Experiment: Automate application deployment using Jenkins

Course outcome: CO1

Deployment was automated end to end: Jenkins polls GitHub for new commits, and after a successful build it deploys the app to /opt/devops-lab1, restarts the systemd service and verifies the health endpoint. A pushed commit (bump app version to 1.0.1) triggered build #2 automatically without any manual action.

Build trigger

- Poll SCM with schedule H/5 * * * * - Jenkins checks GitHub every ~5 minutes
- When a new commit is found, the pipeline is queued automatically
- Post-build behaviour is handled by the Jenkinsfile post section (success/failure messages), artifact archiving and the deploy stage

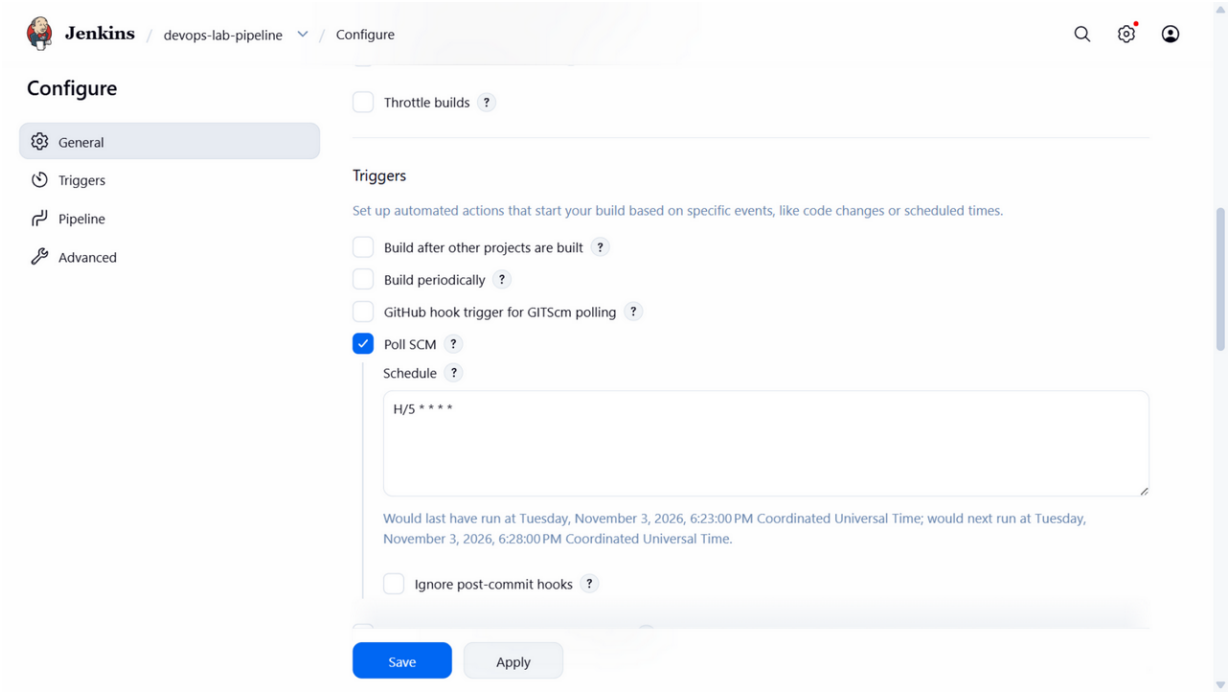


Fig: Poll SCM trigger configuration

Deployment steps

The Deploy stage runs scripts/deploy.sh with passwordless sudo restricted to this one script through /etc/sudoers.d/jenkins-deploy:

```
jenkins ALL=(ALL) NOPASSWD:
    /usr/bin/bash /var/lib/jenkins/workspace/devops-lab-pipeline/scripts/deploy.sh
```

scripts/deploy.sh

```
#!/bin/bash
# Deploys the flask app to /opt/devops-lab1 and restarts the systemd service
set -e

APP_SRC="$(cd "$(dirname "$0")/.." && pwd)"
APP_DEST="/opt/devops-lab1"

echo "Deploying from $APP_SRC to $APP_DEST ..."

sudo mkdir -p "$APP_DEST"
sudo rm -rf "$APP_DEST/app" "$APP_DEST/tests"
sudo cp -r "$APP_SRC/app" "$APP_SRC/tests" "$APP_DEST/"

cd "$APP_DEST"
if [ ! -d venv ]; then
    python3 -m venv venv
fi
./venv/bin/pip install -q -r app/requirements.txt

sudo systemctl restart devops-lab1.service
sleep 3

STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:5000/health || true)
if [ "$STATUS" = "200" ]; then
    echo "Deploy successful - health check returned HTTP $STATUS"
else
    echo "Health check failed (HTTP $STATUS)"
    exit 1
fi
```

/etc/systemd/system/devops-lab1.service

```
[Unit]
Description=DevOps Lab 1 Flask Application
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/devops-lab1
ExecStart=/opt/devops-lab1/venv/bin/python -m flask --app app.app run --host 0.0.0.0 --port 5000
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Automatic deployment evidence (build #2)

Commit 7530c85 (bump app version to 1.0.1) was pushed to GitHub. The next poll found the change and Jenkins started build #2 on its own - the console confirms Started by an SCM change. After the deploy stage the running app reported the new version.

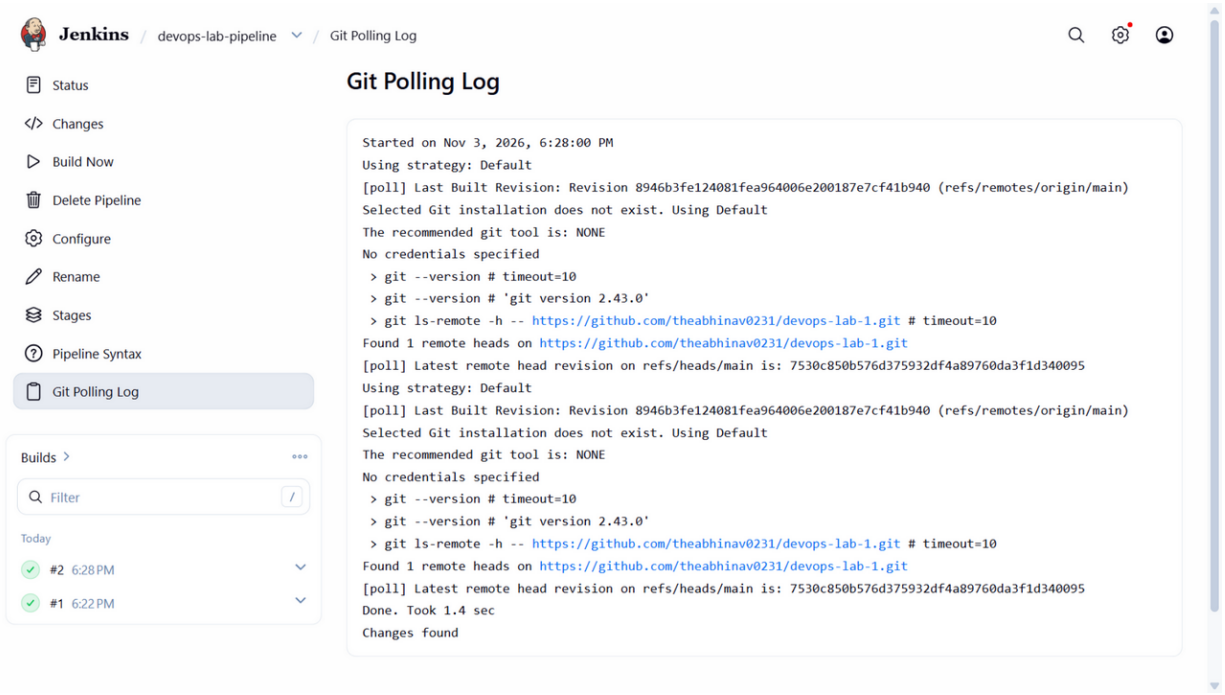


Fig: Git polling log: poll detected the new commit (Changes found)

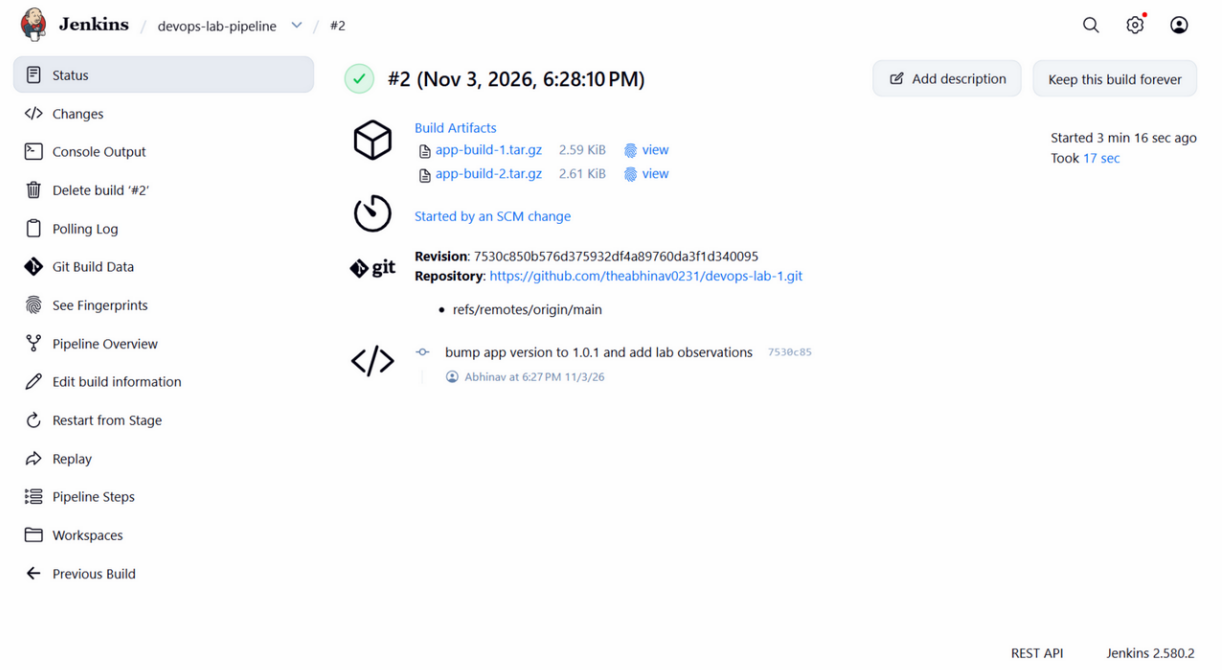


Fig: Build #2 - Started by an SCM change, revision 7530c85

```
$ curl http://localhost:5000/health   (after automatic build #2)
{"status":"UP","version":"1.0.1"}

$ git -C /var/lib/jenkins/workspace/devops-lab-pipeline log -1 --oneline
7530c85 bump app version to 1.0.1 and add lab observations
```

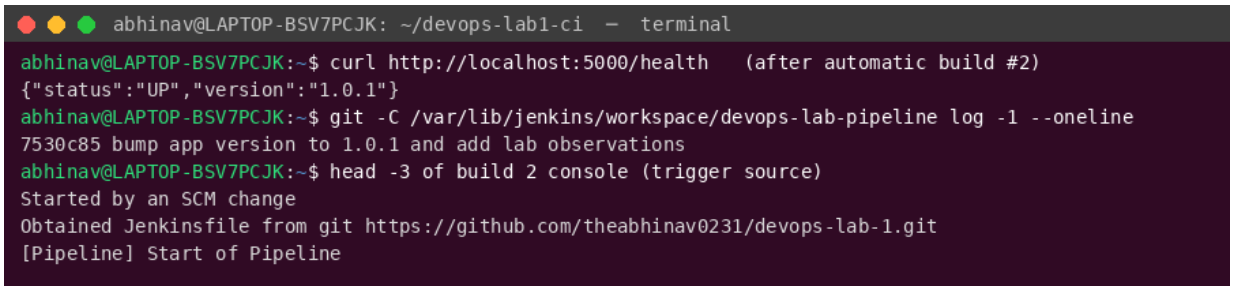


Fig: Deployed app health check after the automatic build

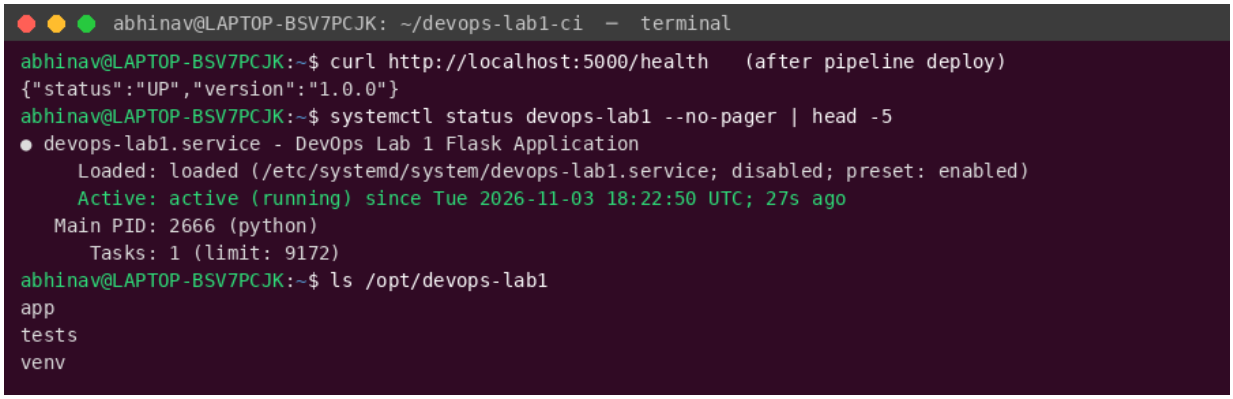


Fig: Service restarted by the pipeline deploy stage

Automation results

Build	Trigger	Result	Time	Deployed version
#1	Build Now (manual)	SUCCESS	29 s	1.0.0
#2	Poll SCM (automatic)	SUCCESS	17 s	1.0.1

Observations

- The deploy script is idempotent - it can run on every build without manual cleanup.
- The health check inside deploy.sh makes a bad deployment fail the pipeline immediately.
- sudo access for the jenkins user is limited to the single deploy script (least privilege).
- No manual step was needed between git push and the new version being served.

Conclusion

All the commands were executed successfully on Ubuntu 24.04 LTS and the evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-1 under the docs/ directory.